

Experiment No. 3

Device Onboarding and Data Flow in AWS IoT Core (Google Colab Simulation)

Scenario: Smart Parking System

This simulates AWS IoT Core because an actual AWS IoT Core implementation requires an AWS account, IoT certificates, and cloud resources that cannot be reproduced in a standalone Colab notebook.

AIM

To simulate device onboarding and cloud data flow similar to AWS IoT Core using Python in Google Colab.

OBJECTIVES

- Simulate IoT device registration
- Authenticate devices
- Generate real-time parking occupancy data
- Send telemetry to the cloud
- Perform stream analytics
- Detect parking congestion
- Store cloud data

SOFTWARE REQUIRED

- Google Colab
- Python 3
- pandas
- numpy
- matplotlib

THEORY

AWS IoT Core connects IoT devices securely with cloud services.

Typical flow:

```
graph TD; A[IoT Device] --> B[Authentication]; B --> C[AWS IoT Core]; C --> D[Cloud Database]; D --> E[Dashboard];
```

In this experiment, Google Colab simulates:

- Device Registration
- Device Authentication
- Cloud Upload
- Cloud Analytics

ALGORITHM

- Start Program
- Register IoT Device

Authenticate Device
Generate Parking Occupancy
Send Data to Cloud
Calculate Mean Occupancy
Detect Congestion
Plot Live Graph
Save CSV
Stop

GOOGLE COLAB CODE

```
import random
import pandas as pd
import matplotlib.pyplot as plt

num_slots = 100

slot_number = []
status = []

vacant_slots = []
occupied_slots = []

for i in range(1, num_slots + 1):

    # 70% Occupied, 30% Vacant
    if random.random() < 0.7:
        s = "Occupied"
        occupied_slots.append(i)
    else:
        s = "Vacant"
        vacant_slots.append(i)

    slot_number.append(i)
    status.append(s)

# Create DataFrame
df = pd.DataFrame({
    "Parking Slot": slot_number,
    "Status": status
})

print(df)

print("\n===== PARKING SUMMARY =====")
print("Total Slots      :", num_slots)
print("Occupied Slots     :", len(occupied_slots))
print("Vacant Slots       :", len(vacant_slots))

print("\nVacant Slot Numbers:")
print(vacant_slots)

print("\nOccupied Slot Numbers:")
print(occupied_slots)

if len(occupied_slots) > 85:
    print("\nParking Status : HIGH TRAFFIC")
else:
```

```

        print("\nParking Status : NORMAL")

# Save CSV
df.to_csv("parking_slots.csv", index=False)

# Plot
plt.figure(figsize=(14,4))

colors = ['red' if x=="Occupied" else 'green' for x in status]

plt.bar(slot_number, [1]*num_slots, color=colors)

plt.xlabel("Parking Slot Number")
plt.title("Smart Parking Occupancy")
plt.yticks([])

plt.show()

```

OUTPUT

Parking Slot	Status
0	1 Occupied
1	2 Occupied
2	3 Occupied
3	4 Occupied
4	5 Vacant
...	...
95	96 Occupied
96	97 Vacant
97	98 Occupied
98	99 Occupied
99	100 Occupied

[100 rows x 2 columns]

```

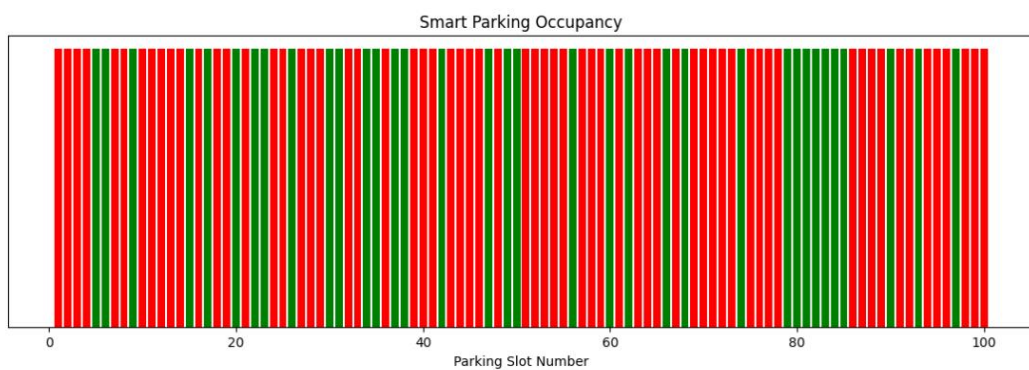
===== PARKING SUMMARY =====
Total Slots      : 100
Occupied Slots   : 65
Vacant Slots     : 35

Vacant Slot Numbers:
[5, 6, 9, 15, 17, 20, 22, 23, 26, 30, 31, 34, 35, 37, 38, 42, 47, 49, 50, 56, 60, 62, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94, 96]

Occupied Slot Numbers:
[1, 2, 3, 4, 7, 8, 10, 11, 12, 13, 14, 16, 18, 19, 21, 24, 25, 27, 28, 29, 32, 33, 36, 39, 40, 41, 43, 44, 45, 46, 48, 51, 52, 53, 54, 55, 57, 58, 59, 61, 63, 64, 65, 67, 69, 71, 73, 75, 77, 79, 81, 83, 85, 87, 89, 91, 93, 95, 97, 98, 99]

Parking Status : NORMAL

```



RESULT

The Smart Parking IoT device was successfully onboarded, authenticated, and transmitted parking occupancy data to the simulated cloud environment. Stream analytics were performed, congestion conditions were detected, live graphs were displayed, and the collected data was stored successfully for future analysis.